

# Intra and Inter-case Features in Predictive Process Monitoring: A Tale of Two Dimensions

Arik Senderovich<sup>1</sup>(✉), Chiara Di Francescomarino<sup>2</sup>, Chiara Ghidini<sup>2</sup>,  
Kerwin Jorbina<sup>3</sup>, and Fabrizio Maria Maggi<sup>3</sup>

<sup>1</sup> Technion - Israel Institute of Technology, Haifa, Israel  
sariks@technion.ac.il

<sup>2</sup> Fondazione Bruno Kessler, Trento, Italy  
{dfmchiara,ghidini}@fbk.eu

<sup>3</sup> University of Tartu, Tartu, Estonia  
{kerwin.jorbina,f.m.maggi}@ut.ee

**Abstract.** Predictive process monitoring is concerned with predicting measures of interest for a running case (e.g., a business outcome or the remaining time) based on historical event logs. Most of the current predictive process monitoring approaches only consider intra-case information that comes from the case whose measures of interest one wishes to predict. However, in many systems, the outcome of a running case depends on the interplay of all cases that are being executed concurrently. For example, in many situations, running cases compete over scarce resources. In this paper, following standard predictive process monitoring approaches, we employ supervised machine learning for prediction. In particular, we present a method for feature encoding of process cases that relies on a bi-dimensional state space representation: the first dimension corresponds to intra-case dependencies, while the second dimension reflects inter-case dependencies to represent shared information among running cases. The inter-case encoding derives features based on the notion of *case types* that can be used to partition the event log into clusters of cases that share common characteristics. To demonstrate the usefulness and applicability of the method, we evaluated it against two real-life datasets coming from an Israeli emergency department process, and an open dataset of a manufacturing process.

**Keywords:** Predictive Process Monitoring · Inter-case Features · Bi-dimensional Feature Encoding

## 1 Introduction

Business processes are supported by process-aware information systems (PAIS) that record execution data into event logs [1]. Predictive process monitoring is concerned with analyzing these event logs to predict measures of interest (MOI) at runtime [2, 14]. Key measures include the business outcome of a process execution (a.k.a. case), the remaining time and the remaining activities up to the

completion of an ongoing case. State-of-the art approaches for solving predictive monitoring problems employ supervised machine learning to make predictions [13, 20]. In particular, event logs are encoded into feature-outcome pairs to build training sets required to run machine learning algorithms [10].

In previous works, predictive process monitoring methods assumed that the MOIs of an ongoing case only depend on intra-case information, e.g., on the execution history of that specific case. This assumption results in encodings that include past events, inter-event durations, and other case attributes [13]. However, the intra-case assumption does not hold in many real-life scenarios. For example, in situations where cases share limited resources, the remaining time of a case heavily depends on other cases that are running at the same time [17].

In this paper, we propose a method for feature encoding in predictive monitoring that relies on a bi-dimensional state space. The first dimension expresses intra-case dependencies, while the second dimension represents inter-case dependencies capturing the interplay of all running cases. However, encoding features into the inter-case dimension is challenging, since (1) there is a large variance in the levels of dependencies for different business processes, and (2) encoding multiple features for a large number of simultaneously running cases may lead to feature space explosion. To overcome these two limitations, we propose an *inter-case encoding* general method that is grounded in the partitioning of recorded (and running) cases into *case types*. Case types represent a flexible instrument to cluster together cases being in the same *state* since the characteristics that define a case state can be different for different scenarios. In addition, to avoid feature space explosion, inter-case features are encoded using a *derivation* function that enables for their compact representation.

We evaluate the proposed method against two real-life datasets coming from an Israeli emergency department (ED), and a manufacturing process. The experiments show that our solution yields a 27% improvement when predicting remaining times of ED patients, and a 52% improvement when predicting the remaining time for the manufacturing process.

The remainder of the paper is structured as follows. We start by motivating our contribution using scenarios taken from a real-life hospital setting that describe different levels of inter-case dependencies (Sect. 2). In Sect. 3, we provide essential background concepts, before stating the main problem that we solve in this paper (Sect. 4). Section 5 presents the core contribution of the paper by specifying the details of our method. An empirical evaluation of the method is presented in Sect. 7, while Sects. 8 and 9 close the paper with related work and conclusive remarks.

## 2 Motivating Scenarios

When considering predictive process monitoring problems in real-life settings, one can identify several scenarios that reflect different levels of inter-case dependencies. Below, we report four realistic scenarios from an ED. The examples are targeting the remaining time prediction as the main MOI.

*Scenario 1: Urgent Patients.* An urgent patient who arrives into the ED requires first aid, and thus receives high priority. Hence, the patient does not compete over shared resources, as she gets immediate help. Here, the dependency between the urgent case and all other cases is negligible, and the remaining time of that case will depend on the clinical history of the patient. This is a case where intra-case features are most predictive for the MOI.

*Scenario 2: Homogeneous Patients.* Consider the distribution of food among patients in the ED. Here, assuming that no patient type has priority over the others, service times are independent and identically distributed for all patients, and the distribution order is random. Hence, we expect that the remaining time for a specific case would depend only on the total number of patients waiting to receive food.

*Scenario 3: Heterogeneous Patient Types with Priorities.* Consider the radiology unit, where patients compete over several types of machines (e.g., XRAY, CT, and MRI). Patients are prioritized according to their diagnosis, age, and recent events. For example, patients who recently went through an Oncology consultation receive the highest priority class. Patients with orthopedic trauma who were recently triaged (seen by a nurse) will receive the lowest priority class. In this setting, it is important to discriminate case types, and consider the number of patients in each priority class [17].

*Scenario 4: Heterogeneous Patient Types with History-Dependent Priorities.* Consider the surgery unit, where patients compete over surgeons and operating rooms. Patients are prioritized according to their diagnosis, age, and their clinical history. For example, patients with a severe diagnosis and who already had other similar suspected diagnosis or other severe diseases in the past, receive the highest priority class. Patients who never had health problems and with a minor trauma will receive the lowest priority class. In this setting, a more fine-grained case typing than in Scenario 3 must be considered. In particular, one needs to take into account the history of the patients when creating priorities.

To summarize, we observe that different processes may require different feature encodings to capture inter-case dependencies. We shall return to these scenarios in Sect. 5, where we present our solution to inter-case feature encoding.

### 3 Preliminaries

In this section, we provide the required preliminaries for our work, namely the definition of event logs, the formulation of the predictive process monitoring problem, and the description of the supervised machine learning framework.

#### 3.1 The Event Log

In what follows, we define an event log, our representation of process execution recordings. Let  $\mathcal{E}$  be the universe of all events. An event log  $L$  is a  $K$ -sized set of event sequences (or cases),  $L = \{\sigma_i : i = 1, \dots, K\}$  with  $\sigma_i = (e_i^1, \dots, e_i^{n_i}) \in \mathcal{E}^*$

being the  $i$ th case of length  $n_i$ . We denote by  $\mathcal{L}$  the universe of event logs. Without loss of generality, we assume that  $L$  comprises complete cases only. However, for running case prediction, we are interested in prefixes of complete cases. Hence, we define the prefix function,  $\phi : \mathcal{E}^* \times \mathbf{N}^+ \rightarrow \mathcal{E}^*$ , which returns a prefix of size  $n$ , given a sequence, namely  $\phi(\sigma_i, n) = (e_i^1, \dots, e_i^n) : n \leq n_i$ . The function returns  $\sigma_i$  in case  $n > n_i$ . We define the extended event log  $L^*$  to be the event log that contains all prefixes of  $L$ , i.e.,  $L^* = \{\phi(\sigma_i, n) : \sigma_i \in L, n \leq n_i\}$ .

Further, we assume that every event in a sequence is associated with data in the form of attribute-value (AV) pairs. Formally, we define the AV function  $\alpha$  to be  $\alpha : \mathcal{E} \rightarrow \mathcal{A}_1 \times \dots \times \mathcal{A}_p$  with  $\mathcal{A}_j, j = 1, \dots, p$  being  $p$  attribute domains. We assume that  $\exists j : \mathcal{A}_j$  is the universe of timestamps (e.g., UNIX time recordings). We denote the domain of timestamps by TS. Moreover, we assume that, for  $j = 1, \dots, p$ ,  $\mathcal{A}_j$  contains the *unknown* value  $\perp$ . Lastly, each prefix of  $\sigma_i$  is associated with a label (e.g., the remaining time for a running case) such that  $y(\phi(\sigma_i, n)) \in \mathcal{Y}$  with  $\mathcal{Y}$  being the domain of labels. Note that the labels can be dynamic and change as the case progresses (e.g., the remaining time for a running case changes as the case progresses).

$\sigma_1 = (\text{consultation}, \{33, \text{radiotherapy}, 10:30\text{AM}\}, 50), (\text{ultrasound}, \{33, \text{nursing ward}, 10:55\text{AM}\}, 25), \dots$   
 $\dots$   
 $\sigma_K = (\text{order blood}, \{56, \text{general lab}, 12:30\text{PM}\}, 120), \dots, (\text{payment}, \{56, \text{financial dept.}, 2:30\text{PM}\}, 0)$

**Fig. 1.** Example of an event log.

As a running example, consider the event log presented in Fig. 1 pertaining to a medical treatment process. Each case relates to a different patient and the corresponding sequence of events indicates the activities executed for a medical treatment of that patient. In the example, *consultation* is the first event of sequence  $\sigma_1$ . Its AV function maps the *consultation* event to “{33, radiotherapy, 10:30AM}” corresponding to the data associated to attributes *age*, *department*, and *timestamp*. The *remaining time* (in minutes) is the label that is present for every event (with 50 min being the remaining time after the first event in the first case, and 25 min being the remaining time after the second event). Note that the value of *age* is static, i.e., it does not change for all the events in a case, while the values of *department*, *timestamp*, and *remaining time* are dynamic, i.e., different for every event.

### 3.2 Predictive Process Monitoring

Having defined our data model, we turn to formulate the problem of predictive process monitoring, following the notation in [13, 14].

**Definition 1 (PPM).** *Given a (possibly running) case  $\sigma_x$ , the predictive process monitoring problem (PPM) is to find a function  $f : \mathcal{E}^* \rightarrow \mathcal{Y}$  that accurately maps  $\sigma_x$  to the corresponding label  $y(\sigma_x)$ .*

Note that differently from previous works on predictive process monitoring, here we do not necessarily observe a single outcome per case. In contrast, every prefix of a running case can have a corresponding label in  $\mathcal{Y}$ . Since historical observations of cases (with all their prefixes and dynamic labels) are provided in  $L^*$ , it is natural to employ supervised learning for solving PPM. Hence, we complete the section by defining the supervised learning framework.

### 3.3 Supervised Learning

Below, we introduce the basic definition of a supervised learning task based on [18], which we later utilize to solve the PPM problem.

Let  $\mathcal{X}$  be some vector-feature space, and let  $\mathcal{Y}$  be the outcome space (we abuse notation on purpose as we consider the same domain as for sequence labels). Further, we denote by  $r$  some nonnegative real-valued risk function  $r : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}^{0+}$  that given a predicted outcome and the real label, returns the risk value (here, high risk corresponds to low accuracy and vice versa). For example,  $r$  can be the squared error risk function. The supervised learning task (SLT) is defined as follows.

**Definition 2 (Supervised Learning Task (SLT)).** *Given the tuple  $\langle \mathcal{S}, F, r \rangle$  with*

- $\mathcal{S} \subseteq \mathcal{X} \times \mathcal{Y}$  *being the training set of independent and identically distributed samples from the distribution of  $\mathcal{X} \times \mathcal{Y}$ ,*
- $F \subseteq \mathcal{Y}^{\mathcal{X}}$  *being the class of learning functions, and,*
- $r$  *being the risk function that we wish to minimize,*

*the supervised learning task (SLT) is to find a function  $\hat{f} \in F$  s.t. for a test observation  $(x_0, y_0) \in \mathcal{X} \times \mathcal{Y}$ , the expected risk is minimized:*

$$\hat{f} = \arg \min_{f \in F} \mathbb{E}[r(\hat{f}(x_0), y_0)]. \quad (1)$$

Note that, in real-life situations, we may observe only  $x_0$ , while  $y_0$  is unobserved. Hence, the true risk can be assessed only in hindsight. In practice, to obtain  $\hat{f}$  that solves the SLT, machine learning methods minimize the empirical risk with respect to the training pairs, i.e., for  $\mathcal{S} = \{(x_i, y_i), i = 1, \dots, K\}$ , we select  $\hat{f}$  such that:

$$\hat{f} = \arg \min_{f \in F} \frac{1}{K} \sum_{i=1}^K r(\hat{f}(x_i), y_i). \quad (2)$$

This approach can lead to over-fitting, which can in turn be avoided by proper selection of  $F$ , the class of learners. In this paper, we avoid over-fitting by selecting well-established supervised learning methods that balance over- and under-fitting.

## 4 Problem Setup

In this section, we present the problem that we solve in this paper, namely the *Sequence-To-feature Encoding Problem* (STEP). The problem arises when we aim at casting the predictive monitoring problem (PPM) into a supervised learning task (SLT). The casting appears straightforward. As solution to the PPM,  $f$ , one may consider using  $\hat{f}$  that results from setting: (i) the training set  $\mathcal{S}$  to be  $L^*$  (all prefixes of historical cases); (ii)  $F$  to be some class of functions (e.g., the set of linear functions); and (iii)  $r$  to be some risk function (e.g., the squared error).

However, the training data in the SLT setting,  $\mathcal{S}$ , is assumed to be a set of independent and identically distributed (i.i.d.) observations of feature-outcome pairs,  $(x_i, y_i)$ . In contrast, the training data that stems from  $L^*$  contains a set of highly dependent prefix-outcome pairs,  $(z_i, y(z_i))$ ,  $z_i = \phi(\sigma_i, n)$  with  $i = 1, \dots, K, n = 1, \dots, n_i$ : any two prefixes of the same case are highly correlated as they represent the same process execution (intra-case dependencies), and every two prefixes that run in the process at the same time potentially share limited resources (inter-case dependencies). Furthermore, the SLT solution function  $\hat{f}$  maps a newly observed feature value to a label, while in the PPM problem, the new value is a (possibly running) case. This leads to the need for transforming sequences into features.

**Problem 1 (Sequence-To-feature Encoding Problem (STEP)).** *Let  $L^*$  be an extended event log that contains all prefixes of the sequences in  $L$ . Solving the STEP problem is to find a function  $g : \mathcal{E}^* \times \mathcal{Y} \times 2^{\mathcal{L}} \rightarrow \mathcal{X} \times \mathcal{Y}$  such that the result of its operation,  $\{g(\sigma_i, y(\sigma_i), L^*)\} \subset \mathcal{X} \times \mathcal{Y}$ , is an i.i.d. sample of feature-outcome from  $\mathcal{X} \times \mathcal{Y}$ .*

Clearly, a STEP solution includes a joint choice of  $g$  and  $\mathcal{X}$ . The STEP has been solved in [8, 13, 14, 20] by various intra-case feature encodings. The main contribution of this paper is a novel solution to STEP that leverages inter-case information, while bounding the feature space size.

## 5 Bi-dimensional STEP Solution

In this section, we show our solution to STEP by means of the construction of a bi-dimensional STEP function  $g_\beta$  that maps every prefix in  $L^*$  and its corresponding label into  $\mathcal{X} \times \mathcal{Y}$  with  $\mathcal{X}$  being a bi-dimensional feature space  $\mathcal{X}_1 \times \mathcal{X}_2$ . The first component of the feature space,  $\mathcal{X}_1$ , captures intra-case dependencies, while the second component,  $\mathcal{X}_2$ , represents inter-case dependencies. The first dimension is defined using existing techniques from [7, 13], while the contribution of this paper is in the definition of  $\mathcal{X}_2$ , and in the construction of  $g_\beta$ . In the remainder, we specify the basic requirements that a STEP solution needs to satisfy. Then, guided by these requirements, we provide a method to encode the intra- and inter-case dimensions.

## 5.1 STEP Requirements

Denote  $\mathcal{S}^\beta$  the set of feature-outcome pairs such that

$$\mathcal{S}^\beta = \{(x_i, y_i) = g_\beta(\sigma_i, y(\sigma_i), L^*) \mid \sigma_i \in L^*\}. \quad (3)$$

We are looking for a STEP solution such that the following requirements hold:

- R1: *Sufficiency*. Let  $\sigma_i$  and  $\sigma_j$  be two different prefixes coming from  $L^*$ . We require that  $(x_i, y_i) = g_\beta(\sigma_i, y(\sigma_i), L^*)$  is independent of  $(x_j, y_j) = g_\beta(\sigma_j, y(\sigma_j), L^*)$ , i.e., the resulting label  $y_i$  depends only on  $x_i$  and not on any other  $(x_j, y_j)$  in  $\mathcal{S}^\beta$ .
- R2: *Accuracy*. When applying the SLT framework with a class of predefined functions  $F$  to the training set that results from  $g_\beta$ ,  $\mathcal{S}^\beta \subset \mathcal{X} \times \mathcal{Y}$ , we require that  $g_\beta$  provides the minimal empirical risk with respect to any other function  $g$  that solves the STEP, i.e., for  $(x_i, y_i) = g(\sigma_i, y(\sigma_i), L^*)$  we require:

$$(\hat{f}, g_\beta) = \arg \min_{f, g} \frac{1}{K} \sum_{i=1}^K r(f(x_i), y_i). \quad (4)$$

- R3: *Compactness*. We require that the dimension of the feature space  $|\mathcal{X}|$  will not exceed the minimal required representation for R1 and R2 to hold.

Requirement R1 assures that  $g_\beta$  provides sufficient information such that for  $(x_i, y_i) \in \mathcal{S}^\beta$ , the label  $y_i$  will depend only on  $x_i$ . This implies that after the encoding, both inter-case and intra-case dependencies required to predict the label  $y_i$  are captured by the corresponding  $x_i$ . We use R1 to justify our decisions when constructing  $g_\beta$ .

Further, requirements R2 and R3 assure that  $\mathcal{X}$  provides an accurate representation of the dependencies, and is compact. In order to demonstrate that requirements R2 and R3 hold for our solution  $g_\beta$ , we evaluate the method by using our STEP solution  $g_\beta$  to solve the PPM based on real-world data (Sect. 7). Below, we briefly outline the solution to the intra-case encoding, i.e., the construction of  $\mathcal{X}_1$ . Then, we focus on the main contribution of our work, namely the inter-case STEP encoding of sequences into  $\mathcal{X}_2$ .

## 5.2 Intra-case STEP Encoding

In this part, we present the construction of  $\mathcal{X}_1$ . Solving this aspect of the STEP is related to machine learning techniques for predicting labels in sequential data [7]. These methods transform training data that comprises correlated sequences into an independent representation of the same sample by adding relevant information to the observations. In particular, we select the *sliding window method* (Sect. 3 in [7]) to encode recent (up-to window size  $w$ ) history of the prefix. We denote by  $g_\beta^{(1)}$  the intra-case component of  $g_\beta$ . Then, for  $\sigma_i = (e_i^1, \dots, e_i^n)$ , we get that

$$g_\beta^{(1)}(\sigma_i, y(\sigma_i), L^*) = ((e_i^{n-w+1}, \dots, e_i^n), y(\sigma_i)), \quad (5)$$

assuming without loss of generality that  $w \leq n$ .

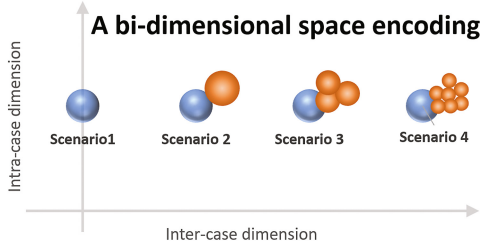
Note that one may consider the encoding of additional static and dynamic attribute-values as shown in [13]. For example, we set  $w = 1$  and choose two AV functions as follows. Let  $\alpha_1 : \mathcal{E} \rightarrow \mathcal{A}_1$  be the elapsed time from the arrival of the patient until every event (dynamic), and  $\alpha_2 : \mathcal{E} \rightarrow \mathcal{A}_2$  be the age of the patient at every event (static if the age of the patient is the same at every event). Then, an encoding of  $\sigma_i$  that considers both AV functions and the last event (since  $w = 1$ ) is:

$$g_{\beta}^{(1)}(\sigma_i, y(\sigma_i)) = (e_i^n, \alpha_1(e_i^n), \alpha_2(e_i^n), y(\sigma_i)). \quad (6)$$

Note that the value of static attributes is constant and hence mentioned only once in encodings involving multiple events. The sliding window approach assumes that the last  $w$  events (and their Attribute-Values) are sufficient to explain the MOI. Therefore, if this assumption holds, requirement R1 is satisfied.

### 5.3 Inter-case STEP Encoding

We now turn to introduce the inter-case STEP encoding. To give intuition, we use Fig. 2, which revisits the four scenarios from Sect. 2.



**Fig. 2.** A graphical representation of our proposed bi-dimensional encodings.

The vertical axis corresponds to the level of dependencies between the predicted label and intra-case features (e.g., recent history, elapsed time, and age), while the horizontal axis captures the level of dependencies for inter-case features (e.g., the number of acute patients in the ED). The blue circle corresponds to the target patient, whose MOI we wish to predict, while the red circles correspond to the patient types on whom the MOI may depend (*case types*). Each case type can be described in terms of intra-case features and the more case types we have, the more fine grained information is required (i.e., the more features are required for encoding this information). In this setting, Scenario 1 is placed on the intra-case axis, as there is no inter-case dimension. In Scenario 2, the intra-case component does not change with respect to Scenario 1, while the inter-case dimension increases in order to take into account all the other patients grouped in the same class (i.e., all of the same case type). Finally, for Scenarios 3 and 4, the inter-case component is further strengthened by means of a more fine-grained case partitioning into types.



The assumption that drives our method is that case types (e.g., urgency priorities) explain the inter-case dependencies between cases. If this assumption holds and the types are properly selected, requirement R1 is satisfied. We also assume that all events are timestamped with an AV function  $\tau$ . The timestamp of a case  $\sigma = (e^1, \dots, e^n) \in L^*$  is  $\tau(\sigma) = \tau(e^n) \in \mathbb{TS}$ . Considering time when encoding inter-case dependencies is crucial, since we assume that these exist only between cases that run in the process at the same time.

Our inter-case STEP encoding relies on four basic concepts, namely *case types*, *discrimination*, *partition*, and (feature) *derivation*. Below, we go over these four concepts:

- *Case Types*. Denote  $\mathcal{T}$  the set of  $m$  case types. Returning to our ED, a type of a patient can be set according to her severity grade.
- *Discrimination*. The discrimination function  $\delta$  is used to set the features that distinguish between case types. In particular, the discrimination function  $\delta(\sigma) \in \mathcal{T}$ , maps a case  $\sigma$  into its type  $T$ . Note that AV functions can be used for discrimination: we may consider, for example, the age of a patient and the elapsed time from the arrival of the patient until the last event. The typed event log is

$$L_T^* = \{(\sigma, T) \subseteq \mathcal{E}^* \times \mathcal{T} \mid \sigma \in L^* \wedge \delta(\sigma) = T\}. \quad (7)$$

- *Partitioning*. In order to categorize cases into types, we define a function  $\pi$  that partitions a typed event log  $L_T^*$  into  $m$  event logs according to their types, namely  $\pi(L_T^*) \in \mathcal{L}^m$ .
- *Derivation*. A derivation function  $\gamma$  maps  $m$  event logs for some time  $t$  into the desired feature space  $\mathcal{X}_2$ , i.e.,  $\gamma : \mathcal{L}^m \times \mathbb{TS} \rightarrow \mathcal{X}_2$ . For example,  $\gamma$  can produce the feature *number of patients of type  $i = 1, \dots, m$  in the ED at time  $t$* .

To demonstrate the specification of the four concepts, consider the encoding of  $\sigma_i = (e_i^1, \dots, e_i^n) \in L^*$  in Scenario 2 (i.e., food distribution in the ED), where all patients are assumed to be homogeneous. Trivially, the case type set is  $\mathcal{T} = \{Patient\}$ , and the discrimination function results in a typed event log  $L_T^* = \{(\sigma, Patient) \mid \sigma \in L^*\}$  as all patients are of the same type ('Patient'). The partitioning function returns  $L_T^*$  itself, while the derivation function  $\gamma$  can be set to the number of cases by type at time  $\tau(\sigma_i)$ :

$$\gamma(L_T^*, \tau(\sigma_i)) = |\{(\sigma, T) \in L_T^* \mid \tau(\sigma) \leq \tau(\sigma_i) \wedge \delta(\sigma) = T\}|. \quad (8)$$

In essence, once the quadruplet  $(\mathcal{T}, \delta, \pi, \gamma)$  is specified, the inter-case component of  $g_\beta$ , which we denote by  $g_\beta^{(2)}$ , is constructed as follows:

$$g_\beta^{(2)}(\sigma_i, y(\sigma_i), L^*) = \gamma(\pi(\{\delta(\sigma) \mid \sigma \in L^*\}), \tau(\sigma_i)). \quad (9)$$

Let us consider the encoding of  $\sigma_i = (e_i^1, \dots, e_i^n) \in L^*$  in a less trivial scenario, Scenario 3, where patients are typed according to their last event (where the event represents the visited unit) into  $m$  urgency types,  $\mathcal{T} = \{e_1, \dots, e_m \mid e_i \in \mathcal{E}\}$

with  $m = |\mathcal{E}|$  and  $e_i$  being the possible events. The discrimination function sets  $\delta(\sigma) = e^n$ , with  $e^n$  being the last event of  $\sigma$ . The partitioning is made according to the typed event log that results from  $\delta$ , and the derivation function remains as in Eq. (8). This leads to  $m$  features: the number of patients of all possible urgencies at time  $\tau(\sigma_i)$ .

Each of the components  $\delta$ ,  $\pi$  and  $\gamma$  plays a different role in our method. The partitioning function  $\pi$  classifies cases according to the notion of similarity imposed by the discrimination function  $\delta$ . To avoid feature space explosion, the derivation function allows an aggregation of the resulting typed event logs. Having fully defined the bi-dimensional STEP solution, we are now ready to test requirements R2 (accuracy) and R3 (complexity) against real-life event logs.

## 6 Implementation

In this section, we introduce the four instances of the STEP framework we have implemented. They take inspiration from the four scenarios presented in Sect. 2. Each of them aims at predicting the remaining time of an ongoing case. We believe indeed that the remaining time is among the most relevant MOIs when dealing with inter-case dependencies.

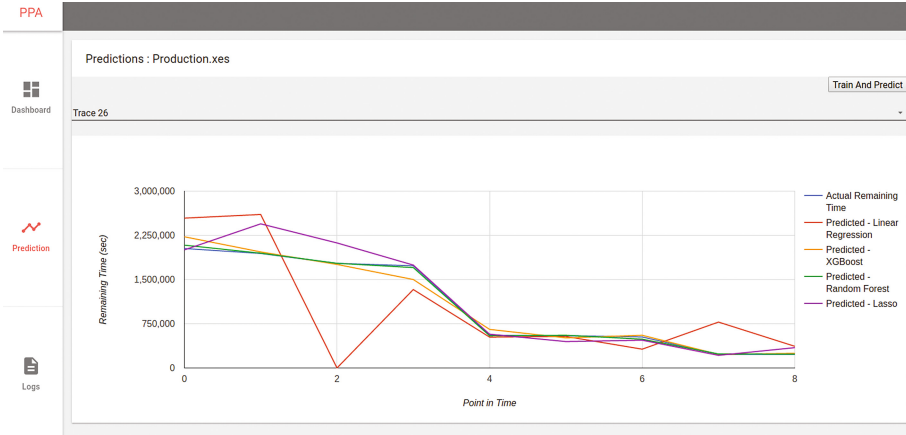
- LEVEL 0. Only the intra-case dimension is taken into account (as in *Scenario 1*) and it is instantiated as described in Sect. 5.2 with a window size of the maximal training trace length.
- LEVEL 1. As in *Scenario 2*, a coarse-grained level of inter-case dependencies is identified. There is only one case type, i.e., a unique priority class, and the derivation function is set to the number of cases running at time  $t$ .
- LEVEL 2. As in *Scenario 3*, a fine-grained level of inter-case dependencies is identified. There are  $m$  case types, each based on the last event carried out, the discrimination function is set to the last event in the case and the derivation function is set to the number of cases for each case type, i.e., with the same last event at time  $t$ .
- LEVEL 3. As in *Scenario 4*, an even more fine-grained level of inter-case dependencies is identified and the inter-case dimension is instantiated so that there are  $m$  case types, each defined according to the last  $w'$  events carried out (in particular, we set  $w' = 3$ ). The discrimination function is set to the last  $w'$  events in the case and the derivation function is set to the number of cases for each case type, i.e., with the same last  $w'$  events at time  $t$ .

All feature encoding algorithms have been implemented in Python and are publicly available.<sup>1</sup> We use the implementations and default parameters of scikit-learn [15] for linear regression, cross-validated Lasso, and random forests. We further use the XGBoost package [5] for cross-validated XGBoost.

Finally, a web interface of our prediction system has been realized in the alpha version of a Predictive Process Analytics (PPA) tool.<sup>2</sup> In particular, the front end

<sup>1</sup> <https://github.com/kerwinjorbina/predictive-process-webservice>.

<sup>2</sup> The web application is available at <http://obscure-springs-12588.herokuapp.com>.



**Fig. 3.** Prediction results in the Predictive Process Analytics (PPA) interface.

application uses the template of Angular Material Dashboard<sup>3</sup> while the back end uses the Python Django Framework<sup>4</sup> to process the logs and provide predictions. Figure 3 reports a screenshot of the web interface showing the remaining time of a trace at specific points in time in a log.

## 7 Evaluation

In this section, we present the main findings we identified by evaluating our method against real-world event logs. In the remainder, we present our experimental setup and procedure, and conclude the section by providing a detailed overview of the results.

### 7.1 Experimental Setup and Procedure

Below, we describe the experimental setup in terms of the supervised learning methods and datasets that were used in the experiments. We then outline the overall experimental procedure and define our evaluation measures.

**Machine Learning Methods.** In our experiments, we relied on two families of learning algorithms: (1) linear models, including linear regression and Lasso; and (2) tree ensemble methods, including random forests and gradient tree boosting. The linear family was chosen as a weak regression baseline, to show that selecting a strong regression method is important for making the method applicable. Below, we briefly describe ensembles of regression trees.

*Ensembles of Regression Trees.* We selected ensembles of decision trees as learning algorithms due to their non-linearity (expressiveness) and their ability to

<sup>3</sup> <https://angularjs.org/>.

<sup>4</sup> <https://www.djangoproject.com/>.

automatically partition the feature space into interpretable regions. In particular, given the set of features, such as age, gender, queue lengths, and recent visit times, the resulting models end up with a clear feature segmentation. Furthermore, using an ensemble rather than a single regression tree typically leads to an improvement in accuracy [3]. We briefly describe the ensemble methods that we consider, namely random forests and XGBoost.

Random forest (RF) [4], is an ensemble built by learning each tree on a different bootstrap replica of the original training set. A bootstrap replica is obtained by randomly drawing (with replacement) original samples and copying them into the replica. Each tree is learned by starting with a single node and greedily extending the tree. Leaves are split using the test (features and values) that maximizes the reduction in quadratic error. The RF algorithm is known not to over-fit as the number of trees increases. We present the results for RF with 50 trees since increasing the number of trees beyond 50 up to 1000 resulted in an improvement of less than 5% in our accuracy measures.

The second ensemble algorithm, eXtreme Gradient Boosting (XGBoost) [5], is the most recent version of the gradient tree boosting algorithm first introduced by Friedman [11]. Unlike RF, instead of weighting the samples, gradient boosting modifies the target variable value for learning each tree.

**Datasets.** Our experiments used two real-world datasets, namely DS1 and DS2.

*DS1.* The data comes from the Electronic Health Record of an Israeli ED that has a maximal capacity of approximately 100 patients per day. Every patient that enters the ED receives a bar-code that gets tracked at the start and end of every medical procedure. Patient treatment events include ‘Reception.End’, ‘Triage.Start’, ‘Treatment.Start’, etc. The events are manually recorded by scanning the patient’s bar-code. The dataset ranges between March 2014 and May 2015, with approximately 42 000 patient visits. The median length-of-stay in the ED is 290 minutes. Lastly, the data comprises approximately 350 000 events coming from the 42 000 cases.

*DS2.* This log published in the IEEE TF on Process Mining site<sup>5</sup> contains a comma separated value dataset in a production process. This provides data on activities, resources, machines and type of items that are manufactured. The dataset occurred from January 2012 to March 2012, with 225 cases. Listed in this log are 55 types of activities, 49 human resources and 31 machines. The median case duration is almost 14 days. The dataset contains approximately 7 200 events.

**Experimental Procedure.** We exploit the training-validation-test paradigm [10] to evaluate our method and randomly partition the two datasets into 60% training data, 20% validation data, and 20% test data. Note that we operate our method to encode inter- and intra-case features prior to dataset partitioning. The encoded data is used as an input to the supervised learning algorithms. While random forests are robust against over-fitting, an increased number of trees raises the computational complexity. Hence, we evaluated the number of

<sup>5</sup> <http://data.4tu.nl/repository/uuid:68726926-5ac5-4fab-b873-ee76ea412399>.

required trees by adding trees and stopping when the improvement is small. The Lasso model was cross-validated to select the regularization parameter. We used Lasso only for Level 3 encoding, as in all other cases linear regression dominated against the validation set. Our experiments were conducted on an 8 core Intel Xeon CPU E5-2660 v4 @ 2.00 GHz each core with 32 GB memory, running on Linux Centos 7.3 OS.

We measure the accuracy of predictions with two empirical loss functions: RMSE and MAE. Root Mean Squared Error (RMSE) is grounded in the squared difference between the actual time and the predicted value. Let  $y_l^*$  be the actual value of  $y_l$ , the time of interest for a log entry of the test set  $l \in L_{test}$ . With  $\hat{y}_l$  being the predicted value, the RMSE is defined as:

$$RMSE = \sqrt{\sum_{l \in L_{test}} [\hat{y}_l - y_l^*]^2}.$$

The RMSE quantifies the error in the time units of the original measurements (in our case seconds, which are converted to minutes and hours in the experiment reporting, for convenience).

The RMSE is sensitive to outliers [10]. Therefore, we also consider the absolute error, which is known to be more robust [10]. In particular, the Mean Absolute Error (MAE) is defined as:

$$MAE = \sum_{l \in L_{test}} |\hat{y}_l - y_l^*|.$$

In the next section, we present these errors for the two datasets and different levels of inter-case feature encoding. We compare them with a purely static baseline, i.e., which takes into account only intra-case features. Furthermore, we plot the RMSE errors for different predicted remaining times.

## 7.2 Results

Table 1 summarizes the results of our experiments. The columns of the table (*Linear*, *RF*, and *XGB*) correspond to the learning methods: linear model (linear regression or Lasso), random forests, and XGBoost, respectively. The rows correspond to the combination of dataset (*DS1* or *DS2*), accuracy measure (*RMSE* or *MAE*), and feature encoding level (*LEVEL0*, *LEVEL1*, *LEVEL2* and *LEVEL3* introduced at the beginning of Sect. 6). The RMSE and MAE are presented in minutes for the ED, and in hours for the production process. In the table, the boldfaced values are the best combination of machine learning method and feature encoding function.

Observing the results, we notice that for both datasets, for all types of encodings and for both error metrics, the best results are obtained by using XGBoost, followed by Random Forests and Linear models. This implies that the remaining times in both datasets are better described by the gradient boosting model. Moreover, by looking at the different levels of encoding, we observe an increase

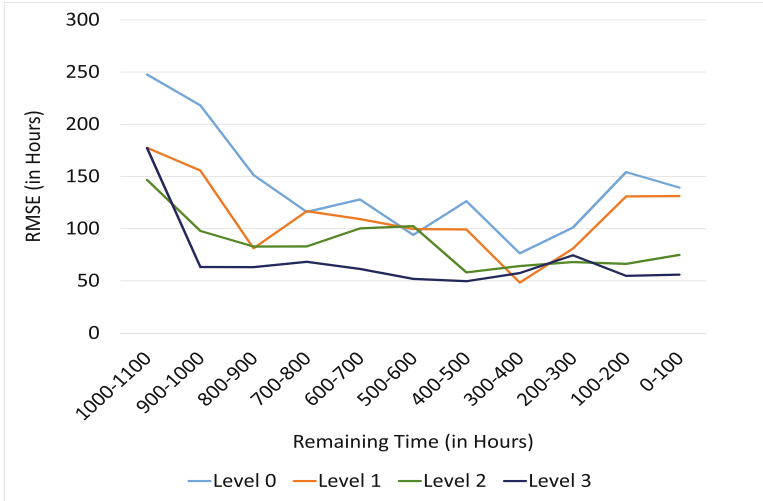
of accuracy for both DS1 and DS2, as more inter-case information is added. This can be explained by the hypothesis that there are strong inter-case dependencies for both datasets. An interesting phenomenon is that case interplay is well-captured by non-linear machine learning techniques (i.e., RF and XGBoost), but not by linear models. This can be explained by a non-linear relation between the remaining time of a running case, and the features coming from the bi-dimensional representation. Overall, we can observe that by taking into account inter-case dependencies, the proposed method increases accuracy of remaining times prediction by 27% in *DS1*, and by 52% in *DS2* with respect to LEVEL0 (intra-case only).

Next, we set to investigate the influence of the predicted value (the remaining time) on the error. Specifically, we chose to focus on the best machine learning method (XGBoost), running on our second dataset (DS2). Figure 4 presents the RMSE as function of the remaining times in the test set. Note that we do not provide the same plot with the MAE, as both present similar trends. Observing the plot, we conclude the following results: (1) as expected, for all levels, accuracy is worse for longer remaining times (e.g., > 1 000 hours); (2) few (or none) inter-case features cause that the prediction accuracy increases as we get more information about the running case, although with some oscillations; (3) more inter-case features guarantee that the accuracy stabilizes faster and remains stable also for short remaining times. For instance, LEVEL3 has almost constantly lower RMSE values compared to the other encodings. This result confirms and strengthens the aggregated results shown in Table 1. Therefore, our method meets requirement R2.

Finally, we report that the training phase in our experiments took from few minutes (for LEVEL0) to about 1 h (for LEVEL3) for *DS1* and from few seconds to 5 min for *DS2*. From this, we can conclude that our method also meets requirement R3.

**Table 1.** Prediction accuracy.

| DS  | Encoding | Measure | Linear | RF  | XGB        |
|-----|----------|---------|--------|-----|------------|
| DS1 | LEVEL0   | RMSE    | 213    | 206 | 176        |
|     |          | MAE     | 142    | 135 | 113        |
|     | LEVEL1   | RMSE    | 212    | 187 | 151        |
|     |          | MAE     | 142    | 123 | 95         |
|     | LEVEL2   | RMSE    | 212    | 177 | 138        |
|     |          | MAE     | 142    | 116 | 85         |
|     | LEVEL3   | RMSE    | 211    | 169 | <b>130</b> |
|     |          | MAE     | 142    | 105 | <b>80</b>  |
| DS2 | LEVEL0   | RMSE    | 256    | 195 | 189        |
|     |          | MAE     | 136    | 72  | 67         |
|     | LEVEL1   | RMSE    | 253    | 158 | 142        |
|     |          | MAE     | 137    | 58  | 52         |
|     | LEVEL2   | RMSE    | 221    | 121 | 97         |
|     |          | MAE     | 137    | 50  | 90         |
|     | LEVEL3   | RMSE    | 216    | 109 | <b>91</b>  |
|     |          | MAE     | 135    | 42  | <b>33</b>  |



**Fig. 4.** XGBoost RMSE errors for different predicted remaining times in DS2.

## 8 Related Work

Predictive process monitoring (PPM) was studied extensively over the past. The methods for solving PPM can be divided into two categories, namely *model-based* and *sequence-to-feature encoding* (STEP). Below, we go over the related work in these two categories.

**Model-Based Solutions to Predictive Monitoring.** In [2], an annotated transition system is mined from an event log. The model is then used to predict remaining times based on remaining times of historical prefixes. The two main limitations of this approach are: (1) the curse of dimensionality since the transition system ‘explodes’ in size as the number of process variants grows, and (2) generalization, since the use of historical remaining times without additional features has a poor predictive power. The curse of dimensionality can be controlled by abstracting the transition system state space representation (e.g., by considering bag of activities instead of sequences). To overcome the second limitation, Senderovich et al. [17] proposed the use of a featured transition system, which includes additional contextual attributes such as the queue length, and the current workload. The featured transition system is then used to transform the data into a feature-outcome representation, and non-linear regression methods are applied. In [16], the authors broaden the variety of machine learning techniques that can be applied to the featured transitions system. The model-based solutions balance the trade-off between dimensionality (too many features) and accuracy (too few features) by reducing the model size first. In encoding-based approaches, this trade-off is struck by using the generalization power of machine learning methods to create a compact number of accurate predictors.

**STEP solutions to Predictive Monitoring.** The seminal work by van Dongen et al. [8] employs non-parametric regression to estimate remaining times. The work lays the ground for all STEP solutions to follow, in that it shows an encoding of an event log into feature-outcome pairs. However, the work is limited to control-flow encoding. The work of Folino et al. [9] proposes to add context features (e.g., resource availability) when predicting processing times. In [14], these approaches are extended and the problem of predictive process monitoring is formalized and generalized. Additional related work suggested a general framework for correlating traces in an event log for prediction of MOIs [12]. However, the authors did not propose a feature space reduction mechanism, and therefore the framework can lead to feature explosion.

Continuing the line of work on encoding sequences into features, Leontjeva et al. [13] introduce the so-called *complex sequence encoding*, which utilizes Hidden-Markov Models to extend control-flow features. Here, separate HMMs are trained for each possible label. Then, the likelihood of a trace prefix to belong to each of these two models is measured. The difference in likelihoods is expressed in terms of odds-ratios, which are then used as features to train the classifier. Further, in [19], a Deep Learning approach that utilizes feature encoding was proposed. The main shortcoming of the STEP-based approaches we outlined above is the assumption that the MOIs depend only on the information payload that accompanies the case for which the prediction is made. However, inter-case dependencies that stem from the interplay between running cases (e.g., competition over scarce resources), may influence the MOIs [21]. In [6], the authors develop a method that is tailored to risk prediction in business process executions, while considering the inter-case dependencies. However, their method does not accommodate for temporal predictions, such as the remaining time of a running case. In our work, we propose a general STEP solution that captures inter-case dependencies for (dynamic) MOIs of an arbitrary type.

## 9 Conclusion

This paper proposes an adaptive method for feature encoding in predictive process monitoring. The method provides possible feature encodings within a bi-dimensional space characterized by both intra- and inter-case features that can be tailored for specific datasets. The proposed inter-case encodings rely on the concept of partitioning an event log and the running cases according to a discrimination function that differentiates between case types. A derivation function is applied to the resulting partition to limit the feature space size, thus preventing the curse of dimensionality. The proposed encodings have been evaluated against two real-life datasets coming from an emergency department and a manufacturing process. Our experiments demonstrated a 27% improvement for the emergency department event log, and up-to 52% improvement for the production log, when predicting remaining times of running cases.

In the future, we plan to investigate other instances of the proposed method by exploring additional case types, as well as new discrimination, partitioning



and derivation functions. Finally, we aim at exploring other dimensions that can affect the future of a running case and their interplay with the intra- and the inter-case ones. One of them is the dimension related to contextual variables, i.e., exogenous variables that are independent of cases but can influence their future such as weather conditions or strikes.

## References

1. van der Aalst, W.M.: *Process Mining: Data Science in Action*. Springer, Heidelberg (2016)
2. van der Aalst, W.M., Schonenberg, M.H., Song, M.: Time prediction based on process mining. *Inf. Syst.* **36**(2), 450–475 (2011)
3. Breiman, L.: Bagging predictors. *Mach. Learn.* **24**(2), 123–140 (1996). <http://dx.doi.org/10.1023/A:1018054314350>
4. Breiman, L.: Random forests. *Mach. Learn.* **45**(1), 5–32 (2001)
5. Chen, T., Guestrin, C.: XGboost: A scalable tree boosting system. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794. ACM (2016)
6. Conforti, R., de Leoni, M., La Rosa, M., van der Aalst, W.M.P., ter Hofstede, A.H.M.: A recommendation system for predicting risks across multiple business process instances. *Decis. Support Syst.* **69**, 1–19 (2015)
7. Dietterich, T.G.: Machine learning for sequential data: a review. In: Caelli, T., Amin, A., Duin, R.P.W., Ridder, D., Kamel, M. (eds.) *SSPR /SPR 2002*. LNCS, vol. 2396, pp. 15–30. Springer, Heidelberg (2002). doi:[10.1007/3-540-70659-3\\_2](https://doi.org/10.1007/3-540-70659-3_2)
8. van Dongen, B.F., Crooy, R.A., van der Aalst, W.M.P.: Cycle time prediction: when will this case finally be finished? In: Meersman, R., Tari, Z. (eds.) *OTM 2008*. LNCS, vol. 5331, pp. 319–336. Springer, Heidelberg (2008). doi:[10.1007/978-3-540-88871-0\\_22](https://doi.org/10.1007/978-3-540-88871-0_22)
9. Folino, F., Guarascio, M., Pontieri, L.: Discovering context-aware models for predicting business process performances. In: Meersman, R., Panetto, H., Dillon, T., Rinderle-Ma, S., Dadam, P., Zhou, X., Pearson, S., Ferscha, A., Bergamaschi, S., Cruz, I.F. (eds.) *OTM 2012*. LNCS, vol. 7565, pp. 287–304. Springer, Heidelberg (2012). doi:[10.1007/978-3-642-33606-5\\_18](https://doi.org/10.1007/978-3-642-33606-5_18)
10. Friedman, J., Hastie, T., Tibshirani, R.: *The Elements of Statistical Learning*. Springer Series in Statistics, vol. 1. Springer, Berlin (2001)
11. Friedman, J.H.: Greedy function approximation: a gradient boosting machine. *Ann. Stat.* **29**(5), 1189–1232 (2001)
12. de Leoni, M., van der Aalst, W.M., Dees, M.: A general process mining framework for correlating, predicting and clustering dynamic behavior based on event logs. *Inf. Syst.* **56**, 235–257 (2016)
13. Leontjeva, A., Conforti, R., Di Francescomarino, C., Dumas, M., Maggi, F.M.: Complex symbolic sequence encodings for predictive monitoring of business processes. In: Motahari-Nezhad, H.R., Recker, J., Weidlich, M. (eds.) *BPM 2015*. LNCS, vol. 9253, pp. 297–313. Springer, Cham (2015). doi:[10.1007/978-3-319-23063-4\\_21](https://doi.org/10.1007/978-3-319-23063-4_21)
14. Maggi, F.M., Di Francescomarino, C., Dumas, M., Ghidini, C.: Predictive monitoring of business processes. In: Jarke, M., Mylopoulos, J., Quix, C., Rolland, C., Manolopoulos, Y., Mouratidis, H., Horkoff, J. (eds.) *CAiSE 2014*. LNCS, vol. 8484, pp. 457–472. Springer, Cham (2014). doi:[10.1007/978-3-319-07881-6\\_31](https://doi.org/10.1007/978-3-319-07881-6_31)

15. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., Duchesnay, E.: Scikit-learn: Machine learning in python. *J. Mach. Learn. Res.* **12**, 2825–2830 (2011)
16. Polato, M., Sperduti, A., Burattin, A., de Leoni, M.: Time and activity sequence prediction of business process instances. *CoRR* abs/1602.07566 (2016)
17. Senderovich, A., Weidlich, M., Gal, A., Mandelbaum, A.: Queue mining for delay prediction in multi-class service processes. *Inf. Syst.* **53**, 278–295 (2015)
18. Shalev-Shwartz, S., Ben-David, S.: *Understanding Machine Learning: From Theory to Algorithms*. Cambridge University Press, New York (2014)
19. Tax, N., Verenich, I., La Rosa, M., Dumas, M.: Predictive business process monitoring with LSTM neural networks. In: Dubois, E., Pohl, K. (eds.) *CAiSE 2017*. LNCS, vol. 10253, pp. 477–492. Springer, Cham (2017). doi:[10.1007/978-3-319-59536-8\\_30](https://doi.org/10.1007/978-3-319-59536-8_30)
20. Teinemaa, I., Dumas, M., Maggi, F.M., Di Francescomarino, C.: Predictive business process monitoring with structured and unstructured data. In: La Rosa, M., Loos, P., Pastor, O. (eds.) *BPM 2016*. LNCS, vol. 9850, pp. 401–417. Springer, Cham (2016). doi:[10.1007/978-3-319-45348-4\\_23](https://doi.org/10.1007/978-3-319-45348-4_23)
21. Verenich, I.: A general framework for predictive business process monitoring. In: *Proceedings of CAiSE 2016* (2016)